Practical:10	Implementation of Kruskal's algorithm using greedy method.
__/___/2026	

Objective:

- To implement Kruskal's algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.
- To understand how a Disjoint Set Union (DSU) detects cycles quickly.
- To analyze the time and space complexity of Kruskal's algorithm.

Program:

```
# Kruskal's Minimum Spanning Tree using Disjoint Set Union (DSU)
n = 6
edges = [(0, 1, 4), (0, 2, 3), (1, 2, 1), (1, 3, 2),
         (2, 3, 4), (3, 4, 2), (3, 5, 7), (4, 5, 6)] # (u, v, weight)

parent = list(range(n))
size = [1] * n

def find(x):
    # follow parents to the root, shortening the path on the way
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False # already in the same set -> cycle
    if size[ra] < size[rb]:
        ra, rb = rb, ra
    parent[rb] = ra # attach smaller set under bigger set size[ra]
    += size[rb]
    return True
```

```
edges.sort(key=lambda e: e[2]) # sort edges by weight
print("KRUSKAL'S ALGORITHM")
print("Sorted edges (u, v, weight):")
for e in edges:
    print(" ", e)

mst = []
total = 0
for step, (u, v, w) in enumerate(edges, 1):
    if union(u, v):
        mst.append((u, v, w))
        total += w
        print(f"Step {step}: edge {u}-{v} (weight {w}) -> ACCEPT")
    else:
        print(f"Step {step}: edge {u}-{v} (weight {w}) -> REJECT (cycle)")
    if len(mst) == n - 1:
        print("MST has n-1 edges, stop.")
        break

if len(mst) != n - 1:
    raise ValueError("Graph is disconnected")

print("Edges in the MST:")
for u, v, w in mst:
    print(f" {u} - {v} : {w}")
print("Total cost of MST =", total)
```

Output:

```
KRUSKAL'S ALGORITHM
Sorted edges (u, v, weight):
   (1, 2, 1)
   (1, 3, 2)
   (3, 4, 2)
   (0, 2, 3)
   (0, 1, 4)
   (2, 3, 4)
   (4, 5, 6)
   (3, 5, 7)
Step 1: edge 1-2 (weight 1) -> ACCEPT
Step 2: edge 1-3 (weight 2) -> ACCEPT
Step 3: edge 3-4 (weight 2) -> ACCEPT
Step 4: edge 0-2 (weight 3) -> ACCEPT
Step 5: edge 0-1 (weight 4) -> REJECT (cycle)
Step 6: edge 2-3 (weight 4) -> REJECT (cycle)
Step 7: edge 4-5 (weight 6) -> ACCEPT
MST has n-1 edges, stop.
Edges in the MST:
  1 - 2 : 1
  1 - 3 : 2
  3 - 4 : 2
  0 - 2 : 3
  4 - 5 : 6
Total cost of MST = 14
```

Analysis:

a) Problem Analysis

A Minimum Spanning Tree connects all V vertices of a weighted graph using V - 1 edges, with no cycle and with the smallest possible total weight. Kruskal's algorithm builds it edge by edge, looking at the whole graph's edge list instead of growing from one vertex.

b) Greedy Approach and Cycle Detection

Kruskal's algorithm first sorts all edges by weight. It then goes through them from the cheapest to the costliest and accepts an edge only if it does not form a cycle with the edges already accepted. It stops when V - 1 edges are chosen.

To check for a cycle quickly we use a Disjoint Set Union. Each vertex starts in its own set. find(x) returns the set leader of x, and union(a, b) joins two sets. If find(u) and find(v) are equal, u and v are already connected, so adding the edge would make a cycle. Path halving and union by size keep these operations almost constant time.

c) Example Analysis

Consider the graph with 6 vertices (0 to 5) and these edges, already sorted by weight:

Edge	Weight
1 - 2	1
1 - 3	2
3 - 4	2
0 - 2	3
0 - 1	4
2 - 3	4
4 - 5	6
3 - 5	7

Trace of the algorithm:

Step	Edge	Weight	Decision	Cost so far
1	1 - 2	1	Accept	1
2	1 - 3	2	Accept	3
3	3 - 4	2	Accept	5
4	0 - 2	3	Accept	8
5	0 - 1	4	Reject (cycle)	8
6	2 - 3	4	Reject (cycle)	8
7	4 - 5	6	Accept	14

Edges 0 - 1 and 2 - 3 are rejected because their end points are already connected (through 0 - 2 - 1 and 2 - 1 - 3). The algorithm stops after 5 accepted edges (V - 1). Total cost of the MST = 14.

d) Algorithm Analysis

Step 1: Create the list of all edges (u, v, weight).

Step 2: Sort the edges in increasing order of weight.

Step 3: Make every vertex its own set (parent[i] = i).

Step 4: Take the next edge. If find(u) is different from find(v), accept it, join the two sets and add its weight to the total.

Step 5: Otherwise reject the edge, because it would form a cycle.

Step 6: Repeat steps 4 and 5 until V - 1 edges are accepted. If the edges run out first, the graph is disconnected.

e) Time Complexity

Sorting the E edges takes O(E log E). The loop then makes at most E find and union calls, which together take almost O(E) time (O(E · α(V)), where α is the very slowly growing inverse Ackermann function). Sorting is the larger part.

Therefore:

Time Complexity = O(E log E), which is the same as O(E log V) because E ≤ VZ

f) Space Complexity

The edge list needs O(E) space, and the parent and size arrays need O(V) space.

Therefore:

Space Complexity = O(V + E)

Conclusion:

Kruskal's algorithm was successfully implemented in Python using sorting and a Disjoint Set Union. For the given graph it accepted 5 edges and rejected the two edges that formed cycles, giving a total MST cost of 14, the same cost as Prim's algorithm on this graph. The time complexity is O(E log E) and the space complexity is O(V + E).